

Summary

This was a very ambitious chapter on the topic of inheritance: the art of “programming by difference.” The basic idea of inheritance is to build a class (a subclass) that incorporates all the methods and instance variables of another class (a base class), thereby allowing you to reuse existing code. Your new subclass can choose to use or override the methods of the base class, as well as define its own methods. A method in a subclass can find the base class by using a call to `super()`.

We built two classes, `InputNumber` and `DisplayMoney` that provide highly reusable functionality. These classes are implemented as subclasses that use classes in the `pygwidgets` package as base classes.

Any client code that uses your subclass will see an interface that incorporates methods defined in both the subclass and the base class. Any number of subclasses can be built using the same base class. An abstract class is one that is not intended to be instantiated by client code, but rather is intended only to be inherited from by subclasses. An abstract method in a base class is one that *must* be overridden in each subclass.

We worked through a number of examples to demonstrate inheritance in the `pygwidgets` package, including how the `TextButton` and `CustomButton` classes both inherit from a common base class, `PygWidgetsButton`.

I showed how you can build a class hierarchy, where a class inherits from another class, which in turn inherits from a third class, and so on.

Inheritance can be complex—reading someone else’s code can be confusing—but as we’ve seen, inheritance can be extremely powerful.